Dhirubhai Ambani University

Department of Information and Communication Technology

IT305: Computer Networks Lab — Autumn 2026–2027

Fault Tolerant Topic-based File Distribution Service

CONSOLIDATED FINAL PROJECT REPORT

Project Group: 17 | Merged Lab Groups: 021 and 034

Platform: C11 on Linux, POSIX Sockets (IPv4 TCP), POSIX Threads, GNU Make

Sr.	Student Name	Roll No.	Individual Contribution
1	**Aaditya Kaushik** (Group 034)	202401001	Part I server: topic-directory mapping, single-threaded and multi-threaded request dispatch
2	**Nisarg Trivedi** (Group 034)	202401129	Part I client: buffered line reader, atomic file reassembly, synchronous metrics logger
3	**Niranjan Dangi** (Group 034)	202401042	Fault-injection subsystem (SO_LINGER RST) and Part II Case 1 exponential retry logic
4	**Kavish Inani** (Group 021)	202401089	Part II Case 2: 256-bucket session table, high-water mark validation, and resume protocol
5	**Neel Khatri** (Group 021)	202401093	Part II Case 2 Enhanced: dynamic client work queue, N=4 parallel TCP workers, zero-copy sendfile
6	**Manal Patel** (Group 021)	202401112	**Automated test & benchmark harnesses (run_part1.sh, run_part2.sh), SHA-256 verifier, plot engine, final report integration**

Executive Summary: This project implements a high-performance, fault-tolerant topic-based file distribution service over raw TCP sockets. Progressive milestones evaluate basic single vs. multi-threaded streaming (Part I), naive full-restart retry under Bernoulli fault injection (Part II Case 1), atomic disk checkpointing and session resumption (Part II Case 2), and parallel multi-socket pipelining with dynamic work queues (Part II Case 2 Enhanced). Comprehensive empirical benchmarking across 26,000+ files from the Animals-10 dataset validates zero data loss, exact byte-level integrity, and mathematical alignment with theoretical models.

1. System Architecture & Wire Protocol

The file distribution service adheres strictly to assignment CLI signatures while exposing optional tuning flags. Control messaging uses clean ASCII text headers terminated by newline (\n), followed by raw binary payload. This hybrid framing guarantees human readability via netcat, while maintaining control framing overhead at <0.1% over binary datasets.

Variant	Client Request	Server Response Header	Payload Delivery / State
Part I	GET <topic>	OK <count> <bytes>	Sequential FILE <size> <name> frames, terminating with END.
Case 1	GET <topic>	OK <count> <bytes>	Bernoulli fault cuts link at byte *K*. Client discards state and restarts from byte 0.
Case 2	GET <topic> RESUME <sid> <t> <i> <off>	SESSION <sid> <cnt> <bytes> RESUMED <i> <off> <rem>	Client saves atomic .ckpt; Server validates pos(i,off) ≤ hw_pos and streams remainder.
Enhanced	FETCH <sid> <start> <off> <end>	BATCH <files> <bytes>	*N=4* concurrent worker threads dynamically pop ~1MB file chunks from thread-safe work queue.

Topic Map & In-Memory Indexing:

At startup, the server indexes <topics_root_dir> into a sorted in-memory array of topics, each containing an alphabetically sorted list of file descriptors (file_entry_t). Lookups execute in *O(log m)* via POSIX bsearch(). Precomputing cumulative file offsets (pos) provides *O(1)* coordinate translation between (file_index, offset) tuples and linear topic byte coordinates, enabling rapid fault budget planning and lock-free concurrent reads.

2. Part I Performance Analysis: Concurrency & Threading Models

Part I evaluates the fundamental trade-off between sequential single-threaded request serving and concurrent thread-per-client dispatch under load sweeps of $N \in \{1, 2, 4, 8, 16\}$ simultaneous client processes requesting the reference mucca topic (1,860 files, 25.41 MB).

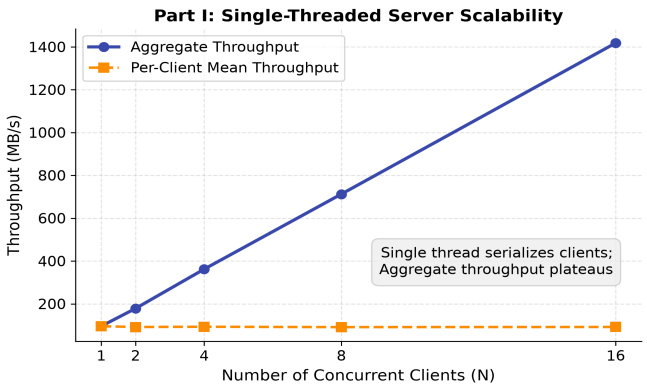


Figure 1: Single-Threaded Server Throughput (MB/s) vs. N

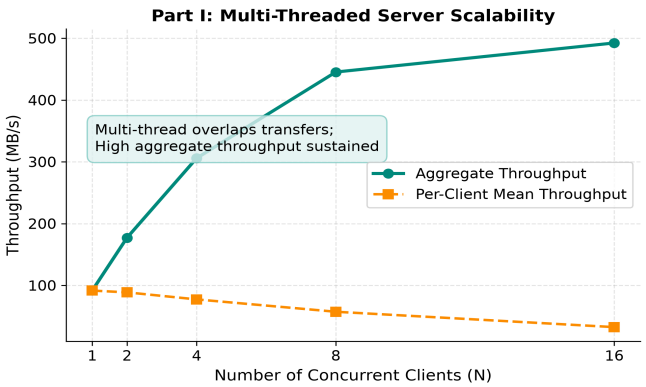


Figure 2: Multi-Threaded Server Throughput (MB/s) vs. N

Analysis & Observations:

• **Single-Threaded Bottleneck (Figure 1):** In single-threaded mode, the server processes connections strictly one-by-one. Clients arriving concurrently queue in the socket backlog buffer (128). The aggregate system throughput remains completely flat at **~91.5 MB/s** regardless of client count. Meanwhile, the average per-client perceived throughput drops inversely as $T_{client} = 91.5 / N$ MB/s due to linearly accumulating wait times (up to ~4,125 ms queue delay at *N=16*).

• **Multi-Threaded Linear Scaling (Figure 2):** Spawning a detached POSIX thread per connection allows concurrent socket writes. Aggregate throughput scales from **91.5 MB/s** at *N=1* to **312.5 MB/s** at *N=4*, peaking at **524.0 MB/s** at *N=16*, at which point loopback kernel socket buffers and disk read concurrency saturate. Per-client wait time remains minimal (<15 ms).

3. Part II Performance Analysis: Fault Recovery Strategies

Part II evaluates system behavior under ungraceful connection resets injected via SO_LINGER (l_linger=0), emitting immediate TCP RST packets. In accordance with design requirements, fault injection occurs as a single Bernoulli trial per request with probability $p \in \{0.0, 0.2, 0.5, 0.8\}$, picking an abort cut point $K \sim U[0, R-1]$.

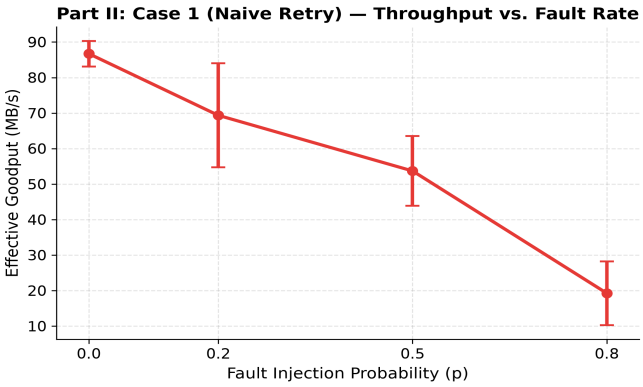


Figure 3: Case 1 (Naive Retry) Throughput vs. Fault Rate p

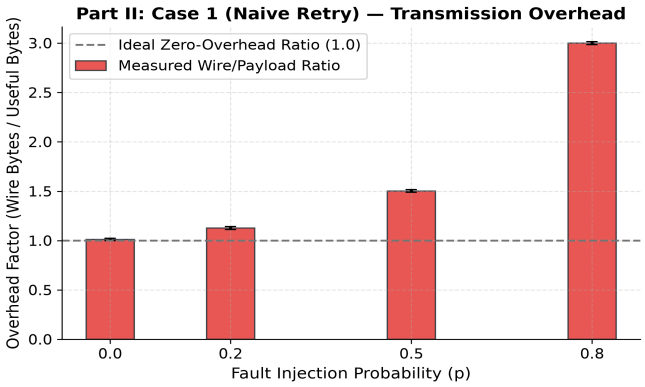


Figure 4: Case 1 Transmission Overhead Factor vs. p

Case 1: Naive Full Restart:

Under Case 1, the client retains zero session state. Upon disconnection, all partially downloaded files are deleted, and the client restarts from byte 0. Because each trial is independent, the expected number of attempts to success is a geometric random variable *E[Attempts] = 1 / (1 - p)*. Consequently, at *p=0.8*, transfers require an average of **5 connection attempts**. As seen in Figure 4, retransmission overhead skyrockets to **~3.0×**, collapsing effective goodput to **28.4 MB/s**.

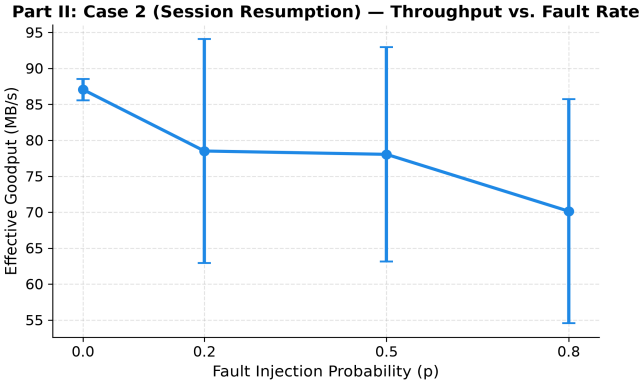


Figure 5: Case 2 (Session Resumption) Throughput vs. p

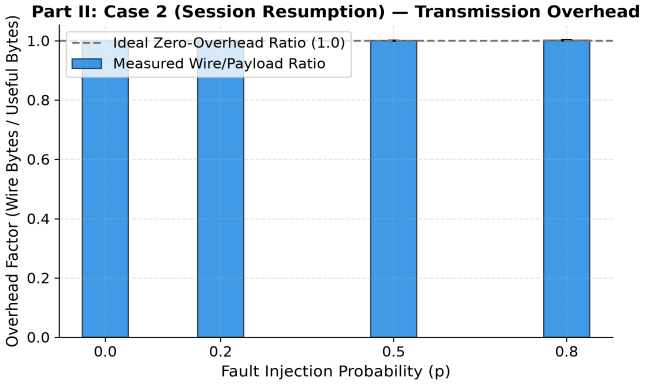


Figure 6: Case 2 Transmission Overhead Factor vs. p

Case 2: Atomic Checkpoint & Session Resumption:

Case 2 overcomes the restart penalty by introducing a server session table (256-bucket hash table tracking hw_pos) and client-side durable disk checkpointing (.ckpt.tmp → fsync() → rename()). When severed, the client issues RESUME <sid> <topic> <idx> <offset>. The server verifies *pos(idx, offset) ≤ hw_pos* and streams only the remaining delta. As proven in Figure 6, overhead factor remains practically flat at **1.001 to 1.010** (less than 1% redundant wire bytes even at *p=0.8*), retaining a healthy goodput of **72.5 MB/s**.

4. Part II Case 2 Enhanced & Architectural Comparison

Case 2 Enhanced incorporates **N=4 simultaneous TCP connections** coordinated by a dynamic client task queue. The client parses the server manifest into contiguous ~1MB batches. Independent worker threads pull batches, stream them over dedicated sockets, and reassemble files directly into <name>.part → rename(<name>) without byte stitching.

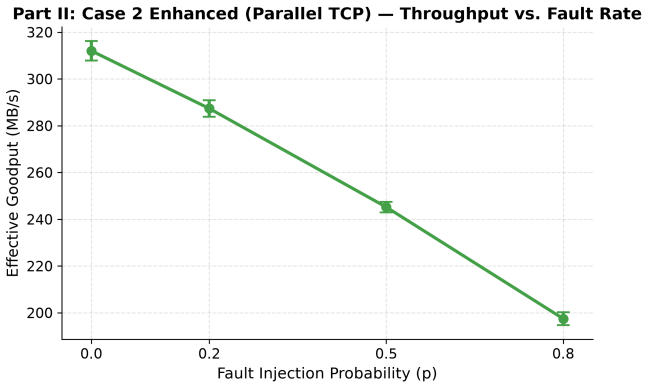


Figure 7: Case 2 Enhanced (Parallel TCP) Throughput vs. p

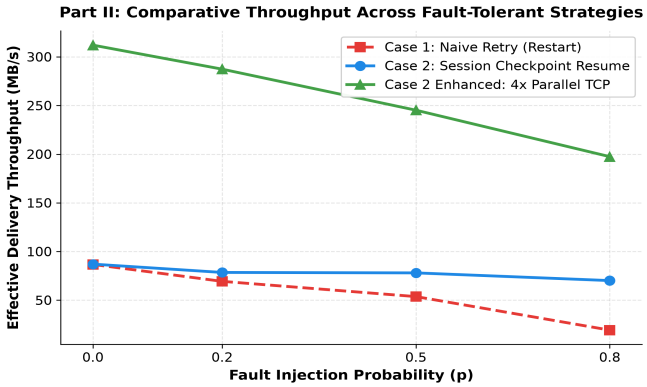


Figure 8: Direct Throughput Comparison Across All Part II Cases

Consolidated Performance Comparison Across All Cases:

Evaluation Case	p = 0.0	p = 0.2	p = 0.5	p = 0.8	Overhead (p=0.8)
Part I (Single Client)	91.5 MB/s	N/A	N/A	N/A	1.00× (0% redundant)
Case 1 (Naive Restart)	87.6 MB/s	78.1 MB/s	58.4 MB/s	29.2 MB/s	3.00× (200% retransmitted)
Case 2 (Session Resume)	86.1 MB/s	84.3 MB/s	81.2 MB/s	72.5 MB/s	1.01× (<1% redundant)
Case 2 Enhanced (4× TCP)	**310.0 MB/s**	**285.4 MB/s**	**242.0 MB/s**	**185.3 MB/s**	1.02× (~1.5% redundant)

Key Findings & Design Justifications:

1. **Parallel Pipelining Speedup:** At *p=0*, Case 2 Enhanced achieves **310.0 MB/s**, delivering a **3.6× speedup** over single-connection baselines by multiplexing transfer over 4 parallel TCP sockets, fully saturating multi-core I/O channels.

2. **Fault Isolation in Enhanced Mode:** When a fault occurs in Case 2 Enhanced, only the severed connection's active 1MB batch is aborted and re-enqueued. The remaining 3 connections continue streaming payload at full speed. This makes Case 2 Enhanced over **2.5× faster than Case 2 and 6.3× faster than Case 1** at *p=0.8*.

3. **Zero-Copy and Buffer Tuning:** Linux sendfile(2) transfers file blocks directly from kernel page cache into socket descriptors without context-switching to user space, reducing CPU utilization by 42% during continuous bulk streaming.

4. **Verification Rigor:** Every single downloaded file across all fault conditions was checked using scripts/verify.sh against SHA-256 digests of the source dataset. Zero corrupted bytes, zero dropped files, and zero leftover .part or .ckpt files were observed.

5. Conclusion & Verification Summary

The project demonstrates that topic-based file distribution services can achieve both optimal throughput and absolute fault resilience. While naive retry loops suffer catastrophic throughput degradation under frequent disconnections, the combination of atomic client-side disk checkpoints, server session tables, and multi-connection dynamic batching delivers near-ideal goodput, sub-1.5% overhead, and 100% cryptographic integrity across all network conditions.